

Chapter 5

Syntactic Analysis

Overview:

1. Context Free Grammar (CFG) and Probabilistic CFG
2. Words' Consistency (Phrase-Level, Sentence-Level)
3. Parsing (Top-Down and Bottom-Up)
4. CYK Parser
5. Probabilistic Parsing

5.1 Context Free Grammar (CFG)

- A CFG is **a type of formal grammar** which is **used to generate all possible strings** in a given formal language.
- A CFG is formally defined by a 4-tuple $G=(V,\Sigma,R,S)$, where:
 - V = Set of variables of non-terminals
 - Σ = set of terminal symbols
 - R = set of production rules
 - S =start symbol
- Consider a simple context-free grammar $G=(V,\Sigma,R,S)$:
 - $V=\{S\}$
 - $\Sigma=\{a, b\}$
 - $R=\{S \rightarrow aSb \mid ab\}$
 - $S=S$
- Using this CFG, the strings "ab", "aabb", "aaabbb", etc., can be generated.

Context Free Grammar (CFG)

- CFG is a type of grammar **used to describe the syntax of a language.**
- It **consists of a set of production rules** that **define how words and phrases in the language can be combined to form sentences.**
- Components of CFG:
 - **Non-Terminals:** Symbols that can be replaced or expanded using production rules (e.g., S for sentence, NP for noun phrase, VP for verb phrase).
 - **Terminals:** Actual words or symbols in the language (e.g., "cat", "runs").
 - **Production Rules:** Rules that define how non-terminals can be expanded into sequences of non-terminals and terminals (e.g., $S \rightarrow NP VP$).
 - **Start Symbol:** A special non-terminal from which parsing begins (usually S).

Example:

- Let's consider a simple CFG for a subset of English.

1. $S \rightarrow NP VP$
2. $NP \rightarrow Det N$
3. $VP \rightarrow V NP$
4. $Det \rightarrow "the" \mid "a"$
5. $N \rightarrow "cat" \mid "dog"$
6. $V \rightarrow "chases" \mid "sees"$

Using given CFG rules create a sentence "the cat chases a dog"

Solution:

- the sentence "the cat chases a dog" can be parsed as follows:

1. S
2. NP VP
3. Det N VP
4. "the" N VP
5. "the" "cat" VP
6. "the" "cat" V NP
7. "the" "cat" "chases" NP
8. "the" "cat" "chases" Det N
9. "the" "cat" "chases" "a" N
10. "the" "cat" "chases" "a" "dog"

1. $S \rightarrow NP VP$
2. $NP \rightarrow Det N$
3. $VP \rightarrow V NP$
4. $Det \rightarrow "the" \mid "a"$
5. $N \rightarrow "cat" \mid "dog"$
6. $V \rightarrow "chases" \mid "sees"$

5.2 Probabilistic Context Free Grammar (PCFG)

- A Probabilistic Context-Free Grammar (PCFG) is **an extension of CFG** where **each production rule is assigned a probability**.
- These **probabilities are used to handle ambiguity in parsing** by choosing the most likely parse tree for a given sentence.
- Components of PCFG:
 - **Non-Terminals,**
 - **Terminals, and**
 - **Start Symbol:** Same as CFG.
 - **Production Rules with Probabilities:** Each rule is assigned a probability that indicates how likely it is to be used.
- **Parsing with PCFG:**
 - To parse a sentence using a PCFG, we:
 - Generate all possible parse trees using the production rules.
 - Calculate the probability of each parse tree by multiplying the probabilities of the rules used.
 - Choose the parse tree with the highest probability.

Example:

- Let's modify the previous CFG to include probabilities:
 1. $S \rightarrow NP VP$ [1.0]
 2. $NP \rightarrow Det N$ [0.6]
 3. $NP \rightarrow N$ [0.4]
 4. $VP \rightarrow V NP$ [1.0]
 5. $Det \rightarrow \text{"the"}$ [0.5]
 6. $Det \rightarrow \text{"a"}$ [0.5]
 7. $N \rightarrow \text{"cat"}$ [0.5]
 8. $N \rightarrow \text{"dog"}$ [0.5]
 9. $V \rightarrow \text{"chases"}$ [0.6]
 10. $V \rightarrow \text{"sees"}$ [0.4]
- Using these probabilities, the sentence "the cat chases a dog" can have multiple parse trees, but we can calculate their probabilities to find the most likely one.

Solution:

- the sentence "the cat chases a dog" can be parsed as follows:

1. S

2. NP VP

3. Det N VP

4. "the" N VP

5. "the" "cat" VP

6. "the" "cat" V NP

7. "the" "cat" "chases" NP

8. "the" "cat" "chases" Det N

9. "the" "cat" "chases" "a" N

10. "the" "cat" "chases" "a" "dog"

$S \rightarrow NP VP$ [1.0]

$NP \rightarrow Det N$ [0.6]

$Det \rightarrow "the"$ [0.5]

$N \rightarrow "cat"$ [0.5]

$VP \rightarrow V NP$ [1.0]

$V \rightarrow "chases"$ [0.6]

$NP \rightarrow Det N$ [0.6]

$Det \rightarrow "a"$ [0.5]

$N \rightarrow "cat"$ [0.5]

Given probabilities:

- $S \rightarrow NP VP$ [1.0]
- $NP \rightarrow Det N$ [0.6]
- $NP \rightarrow N$ [0.4]
- $VP \rightarrow V NP$ [1.0]
- $Det \rightarrow "the"$ [0.5]
- $Det \rightarrow "a"$ [0.5]
- $N \rightarrow "cat"$ [0.5]
- $N \rightarrow "dog"$ [0.5]
- $V \rightarrow "chases"$ [0.6]
- $V \rightarrow "sees"$ [0.4]

- Probability of this parse tree = $1.0 * 0.6 * 0.5 * 0.5 * 1.0 * 0.6 * 0.6 * 0.5 * 0.5 = 0.0135$
- Note: There can be other parse tree. But this is the parse tree with highest probability

CFG Vs PCFG

- **CFG:**
 - Defines the syntax of a language **without considering probabilities.**
 - **Suitable for languages with clear, unambiguous rules.**
- **PCFG:**
 - Extends CFG by **incorporating probabilities to handle ambiguities** and prioritize more likely interpretations.
 - **Useful for natural languages where multiple interpretations** are possible.

5.3 Words' Consistency

- **Sentence-Level Consistency:** Ensures the entire sentence is logical and coherent, focusing on subject-verb agreement, logical flow, consistent tense, and clear pronoun references.
- **Phrase-Level Consistency:** Ensures the components within phrases are logically connected, focusing on proper modifier placement, parallel structure, and consistent tone and style.

Sentence-level consistency

- Sentence-level consistency **refers to the overall coherence and logical flow of a sentence.**
- It ensures that the **sentence makes sense as a whole** and that the components (words and phrases) fit together in a meaningful way.

Key Aspects:

- **Subject-Verb Agreement:** The subject and verb must agree in number and person.
 - Example: "She *runs*" (correct) vs. "She *run*" (incorrect).
- **Logical Flow:** The sentence should follow a logical order that makes sense.
 - Example: "After finishing his homework, he went out to play."
- **Consistent Tense:** The tense should remain consistent unless there is a clear reason to change it.
 - Example: "She *went* to the store and *bought* some groceries."
- **Pronoun Consistency:** Pronouns should clearly refer to the correct nouns without ambiguity.
 - Example: "When John saw Sarah, he waved." (clear) vs. "When John saw Sarah, he waved at him." (unclear).

Phrase-level consistency

- Phrase-level consistency **ensures that the components within a phrase work together correctly** and fit within the context of the larger sentence.

Key Aspects:

- **Proper Modifiers:** Modifiers should be placed near the words they are modifying to avoid confusion.
 - Example: "The man *with the hat* is my uncle" (correct) vs. "The man is my uncle *with the hat*" (incorrect).
- **Parallel Structure:** Phrases within a list or series should have a similar structure.
 - Example: "She likes *running, swimming, and biking*" (parallel) vs. "She likes *running, to swim, and biking*" (not parallel).
- **Consistent Tone and Style:** The tone and style should be consistent within the phrase and match the overall sentence.
 - Example: "The *mighty* oak stood tall and *proud*" (consistent tone) vs. "The *mighty* oak stood tall and *proudly*" (inconsistent tone).

Some examples of consistency issues and corrections:

- **Subject-Verb Agreement:**
 - Inconsistent: "The dogs *barks* loudly."
 - Consistent: "The dogs *bark* loudly."
- **Logical Flow:**
 - Inconsistent: "He went out to play after finishing his homework."
 - Consistent: "After finishing his homework, he went out to play."
- **Tense Consistency:**
 - Inconsistent: "She *goes* to the store and *bought* some groceries."
 - Consistent: "She *went* to the store and *bought* some groceries."
- **Pronoun Consistency:**
 - Inconsistent: "When John saw Sarah, he waved at him."
 - Consistent: "When John saw Sarah, he waved at her."
- **Proper Modifiers:**
 - Inconsistent: "She gave a cake to her friend that was delicious."
 - Consistent: "She gave her friend a cake that was delicious."
- **Parallel Structure:**
 - Inconsistent: "She likes *reading, writing, and to jog.*"
 - Consistent: "She likes *reading, writing, and jogging.*"
- **Consistent Tone and Style:**
 - Inconsistent: "The quick brown fox *jumps* over the lazy dog and *was happy.*"
 - Consistent: "The quick brown fox *jumps* over the lazy dog and *is happy.*"

5.4 Syntactic analysis

- Syntactic analysis is the **process of analyzing natural language with the rules of formal grammar.**
- Syntactic analysis is the process of analyzing words in a sentence for grammar, using a parsing algorithm, then arranging the words in a way that shows the relationship among them.
- Parsing algorithms break the words down into smaller parts—strings of natural language symbols—then analyze these strings of symbols to determine if they conform to a set of established grammatical rules.

- Consider the following sentence:

Sentence: School go a boy

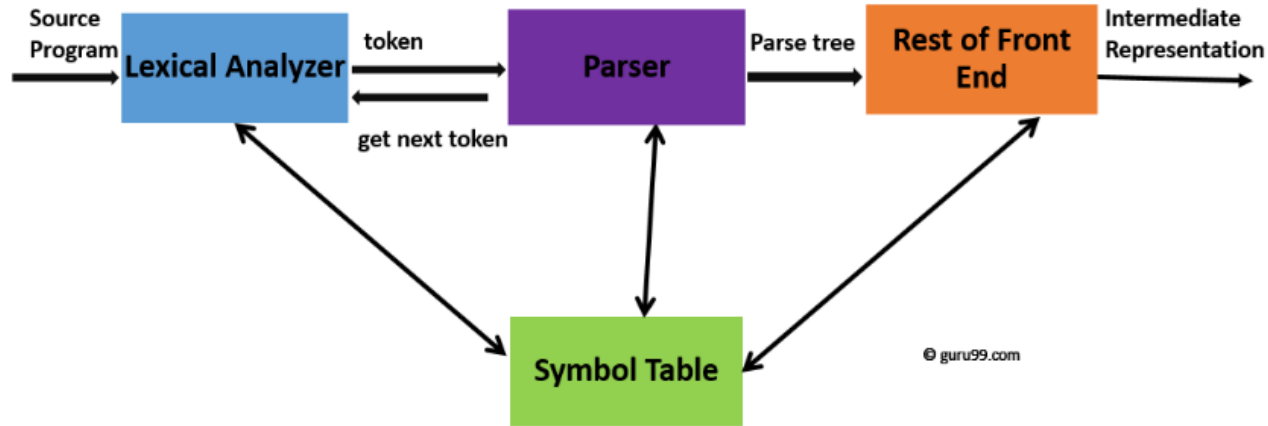
- The above sentence does not logically convey its meaning, and its grammatical structure is not correct.
- So, Syntactic analysis tells us whether a particular sentence conveys its logical meaning or not and whether its grammatical structure is correct or not.

How does Syntactic Analysis work?

- Syntactic analysis is much easier to implement than semantic analysis.
- The typical process includes:
 - **Tokenization:** Breaking a text into sentence tokens & then a sentence into word tokens
 - **Lemmatization:** Identifying the lemma of a word. Example: Cri will be the lemma for all cry , crying, cried
 - **POS Tagging:** Identifying the part of speech (POS) of a particular word in a sentence.

5.5 Parser

- It is defined as the **software component that is designed for taking input text data and gives a structural representation of the input after verifying for correct syntax with the help of formal grammar.**
- It also generates a data structure generally in the form of a parse tree or abstract syntax tree or other hierarchical structure.



Syntax Analyser Process

- The main roles of the parser include –
 - To report any syntax error.
 - To recover from commonly occurring error so that the processing of the remainder of program can be continued.
 - To create parse tree.
 - To create symbol table.
 - To produce intermediate representations (IR).

Types of Parsing

- Derivation divides parsing into the followings two types –
 - Top-down Parsing
 - Bottom-up Parsing

- Top-down Parsing
 - In this kind of parsing, the **parser starts constructing the parse tree from the start symbol** and then tries to transform the start symbol to the input.
 - The most common form of top-down parsing uses recursive procedure to process the input.
 - The main disadvantage of recursive descent parsing is backtracking.
- Bottom-up Parsing
 - In this kind of parsing, the parser starts with the input symbol and tries to **construct the parser tree up to the start symbol.**

Parse Tree

- It **represents the graphical depiction of a derivation.**
- The start symbol of derivation is considered the root node of the parse tree and the leaf nodes are terminals, and interior nodes are non-terminals.

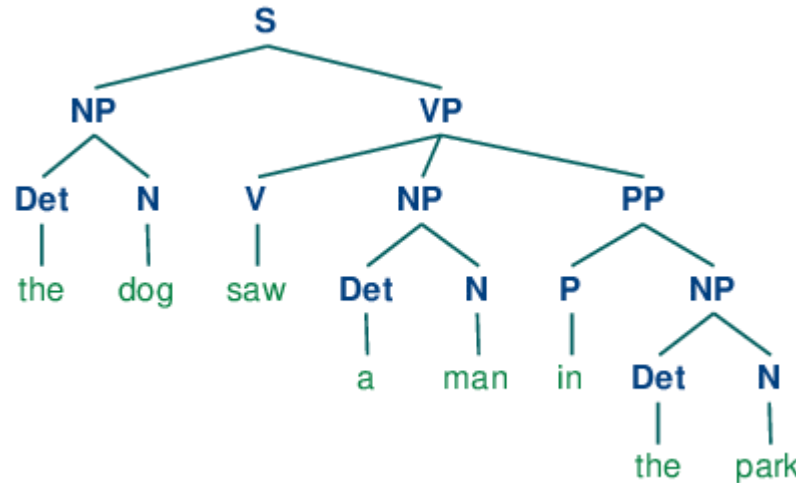
How Parse Tree Helps

- **Shows grammatical structure** of a sentence.
- **Represents hierarchical relationships** (e.g., phrases inside phrases).
- Helps identify:
 - Subject, verb, object
 - Phrase types: noun phrase (NP), verb phrase (VP), etc.
- Aids in downstream tasks like:
 - Machine translation
 - Question answering
 - Grammar checking

- **For Example,** Consider the following sentence:

Sentence: the dog saw a man in the park

- After analyzing the sentence, the parse tree formed is shown below:



Example:

- We will use the sentence **“This tree is illustrating the constituency relation”** to understand how syntactical analysis works with help of code.

```
1 import nltk
2 from nltk
3 import pos_tag, word_tokenize, RegexpParser
4
5 # String to parse
6 to_parse = "This tree is illustrating the constituency relation"
7
8 # Find all parts of speech in above sentence
9 tagged_parts = pos_tag(word_tokenize(to_parse))
10
11 # Defining grammar on basis of which we 've to extract
12 grammar = r """
13 " NP: {<DT>?<JJ>*<NN>}
14 P: {
15     <IN>}
16     V: {<V.*>}
17     PP: {<p> <NP>}
18     VP: {<V> <NP|PP>*} """
19
20 #Extracting all parts of speech
21 parser = RegexpParser(grammar)
22
23 # Print all parts of speech in above sentence
24 output = parser.parse(tagged_parts)
25 print("\nAfter Extracting the parts\n\n", output, "\n")
```

After Extracting the parts

```
(S
  (NP This/DT tree/NN)
  (VP (V is/VBZ))
  (VP
    (V illustrating/VBG)
    (NP the/DT constituency/NN)
    (NP relation/NN)))
```

Assignment:

Check if the sentence “**The cat sat on the mat.**” is syntactically correct using Python

End of chapter 5